Online PES Decoder Learning in Neuromorphic Residual UAV Attitude Control

Equation-to-code companion and audit record
for the ACC 2027 manuscript (version of September 30, 2026)

Gandhi Contreras, Steven Sandoval, and Luis Rodolfo Garcia Carrillo

September 30, 2026

Abstract

This document lets a reader check, equation by equation, that the manuscript describes the code that produced its results. For each of the paper's equations (1)–(30) and both propositions it gives the statement, a short derivation with its standard sources, the exact lines of the implementation that compute it, and a status tag. Section 2 summarizes the machine checks: all 59 numbers the verifier extracts from the paper are recomputed from the raw run outputs (59/59 agree), and every source and model file hash recorded by the published runs equals the file quoted here (9/9).

Contents

1 How to read this document **3**

2 Audit summary **4**

3 Quaternions, kinematics, and attitude error: P-(1)–P-(8) **4**
3.1 Hamilton product and rotation 4
3.2 Active and passive kinematics P-(1), P-(3) 5
3.3 Error quaternion, sign selection, and angle P-(4)–P-(6) 5
3.4 Error kinematics P-(7)–P-(8) 6
3.5 References and the desired rate $\boldsymbol{\omega}_d$ 6

4 Rigid body, inertia, and the torque box: P-(2), P-(9) **7**

5 Allocation and the height loop: P-(10)–P-(12) **8**
5.1 Allocation matrix P-(10) 8
5.2 Clipped inverse allocation P-(11) and the CF2 rotor limit 8
5.3 Collective thrust P-(12) 9

6 Classical baselines: P-(13)–P-(15) **9**
6.1 Gain design P-(13) 9
6.2 Fixed PD P-(14) (Table II, lowest blocks) 10
6.3 PD with desired-rate feedforward P-(15) and the ρ scaling 10
6.4 Other control families (Sec. VI-B, portability) 11

7 The CMG unit: P-(16)–P-(22) 11
7.1 Paired-current encoding P-(16) . . . 11
7.2 Continuous dynamics P-(17)–P-(19) and the membrane step P-(22) . . . 12
7.3 Gate step P-(20)–P-(21) . . . 12
7.4 Bounded states (Proposition 1(i)) . . . 13
7.5 Rest point, resting activity, latching, and χ (Sec. VI-C) . . . 13

8 Readout, PES, residual, and the gate: P-(23)–P-(28) 13
8.1 Readout P-(23) . . . 13
8.2 PES update P-(24)–P-(26) . . . 14
8.3 Bounded residual P-(27) and the saturation-aware gate P-(28) . . . 15
8.4 Proposition 1, full proof . . . 16

9 Proposition 2 and the latched regime: P-(29)–P-(30) 16
9.1 Proposition 2, proof, and the clip lemma . . . 16
9.2 Constant-basis recursion P-(30) . . . 16
9.3 Pole, corner frequency, DC gain . . . 16

10 Neuron-basis comparison (Sec. VI-D) 17
10.1 Equal learning rates across neuron families . . . 17
10.2 Metrics used in Sec. VI-D . . . 18

11 Protocol and metrics (Sec. VI-A, VI-E) 18
11.1 Error metric and reductions . . . 18
11.2 Measurement noise . . . 19
11.3 Repeated disturbances . . . 19

12 Changes relative to the September 12 companion 19

A Source files and hashes 20

B Recomputed paper numbers 21

1 How to read this document

- **Paper references.** "P-(14)" is equation (14) of the manuscript; Proposition 1, Table II and Sec. VI-B refer to the manuscript likewise. Unprefixed numbers refer to this document.
- **Code.** Every listing is read directly from the repository file at the quoted line range, with the file's own line numbers. Paths are relative to the repository root. Appendix A gives the SHA-256 of every cited file.
- **Status tags.** *Standard*: a textbook result, cited. *Derived here*: the steps are written out. *Verified*: the number is recomputed by a script named in the text. *Implementation note*: a detail of the code that the paper leaves implicit; each note says whether it changes any reported number.
- **Which code produced the numbers.** The noise-free, noise and disturbance results at $\rho = 3$ come from `experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` (X2) and its copy `run_a2_robustness.py` (CF2, $\beta = .2$). Both call the shared modules listed in Table 1. The $\rho = 1$ and fixed-PD ladder used the archived runners in `experiments/nengo_nef_replication_2026-08-29/`; the controller-family study used `run_controllers_gated.py`; the neuron-basis study used `experiments/neuron_basis_decisive_2026-09-24/run_neuron_basis.py`. `REPRODUCE_ACC2027.md` in the repository root maps every claim to its script and output file.

Table 1: Shared modules and what they implement.

Module	Implements
`src/gra_control/math3d.py`	quaternion algebra, error, angle, mission references (P-(3)–P-(6))
`src/gra_control/platforms/drone.py`	allocation matrix, torque box (P-(10), $\boldsymbol{\ell}$)
`src/gra_control/gate.py`	direct NumPy CMG–PES controller (P-(16)–P-(27))
`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`	Nengo CMG neuron and PES decoder (P-(16)–P-(27)), used for all reported runs
`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`	`AdvancedBaseline`: PD+ff (P-(15)) and the five other control families; $\boldsymbol{\omega}_d$
`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py`	plant loop: sensing and noise, gate P-(28), allocation P-(11), collective P-(12), disturbance, metrics

2 Audit summary

Table 2: Machine checks behind this document. Scripts are in the repository; each prints PASS/FAIL or the number shown.

Check	Result	Script / record
Paper numbers recomputed from raw run outputs	59/59 agree	`reproduce/verify_acc2027_claims.py`; Appendix B
Cited source and model files vs. hashes recorded by the published runs	9/9 identical	run `manifest.json`; Appendix A
New runner vs. archived Table II $\rho = 3$ block	worst relative error 5.0×10^{-15}	`run_validation.py`
Nengo vs. direct NumPy CMG–PES (ten cases)	worst relative mean error 2.013×10^{-6}	`verify_archived_equivalence.py`
Runner's reconstruction of the residual vs. the Nengo augmentor's command (15 primary CMG runs)	worst 5.4×10^{-20} N m	`summary.json`, field `max_native_vs_reconstructed_command_error_Nm`
Exact recovery at $\beta = 0$ (Prop. 1) for LQR, SMC, MPC, NMPC	relative difference 0 (5 cases)	`certify_beta0_recovery.py`
Backstepping (model feedforward off) $\equiv$ PD+ff P-(15)	20/20 cells bit-identical	`run_campaign_a5_backstepping.py`
Reference rate is the $\boldsymbol{\omega}_d$ of P-(7)–P-(8)	residual $\leq 4 \times 10^{-9}$	`reproduce/check_rate_frame.py`

Software. Python 3.13.11, NumPy 2.5.2, MuJoCo 3.12.0 and Nengo 4.1.0. The primary runs (2026-09-21) record repository commit `67e7e03f1dba`. Every run directory holds `trace.csv`, `summary.json` and a `manifest.json` with the configuration hash, source hashes and library versions.

3 Quaternions, kinematics, and attitude error: P-(1)–P-(8)

3.1 Hamilton product and rotation

For scalar-first $p = [p_0, \boldsymbol{p}^\mathsf{T}]^\mathsf{T}$, $q = [q_0, \boldsymbol{q}^\mathsf{T}]^\mathsf{T}$,

$$p \otimes q = \begin{bmatrix} p_0 q_0 - \boldsymbol{p}^\mathsf{T}\boldsymbol{q} \\ p_0\boldsymbol{q} + q_0\boldsymbol{p} + \boldsymbol{p} \times \boldsymbol{q} \end{bmatrix}, \qquad q^* = [q_0, -\boldsymbol{q}^\mathsf{T}]^\mathsf{T}, \qquad q^{-1} = q^* \text{ for } \|q\| = 1, \tag{1}$$

and a unit quaternion rotates vectors by $R(q)\boldsymbol{v} = \mathrm{vec}(q \otimes [0, \boldsymbol{v}^\mathsf{T}]^\mathsf{T} \otimes q^*)$ [1, 2].

Status. Standard.

`src/gra_control/math3d.py`: 12–25 conjugate and Hamilton product (1)

```python
def quat_conj(q: np.ndarray) -> np.ndarray:
    q = np.asarray(q, dtype=float)
    return np.array([q[0], -q[1], -q[2], -q[3]], dtype=float)


def quat_mul(p: np.ndarray, q: np.ndarray) -> np.ndarray:
    p0, p1, p2, p3 = p
    q0, q1, q2, q3 = q
    return np.array([
        p0*q0 - p1*q1 - p2*q2 - p3*q3,
        p0*q1 + p1*q0 + p2*q3 - p3*q2,
        p0*q2 - p1*q3 + p2*q0 + p3*q1,
        p0*q3 + p1*q2 - p2*q1 + p3*q0,
    ], dtype=float)
```

3.2 Active and passive kinematics P-(1), P-(3)

With Q the body-to-inertial quaternion and $\boldsymbol{\omega}$ the body rate, $\dot{Q} = \frac{1}{2} Q \otimes \tilde{\boldsymbol{\omega}}$, $\tilde{\boldsymbol{\omega}} = [0, \boldsymbol{\omega}^\mathsf{T}]^\mathsf{T}$ [2]. For the passive $q = Q^{-1} = Q^*$, differentiate $Q \otimes Q^* = 1$:

$$\dot{q} = (\dot{Q}^*) = -Q^* \otimes \dot{Q} \otimes Q^* = -\tfrac{1}{2}\, Q^* \otimes Q \otimes \tilde{\boldsymbol{\omega}} \otimes Q^* = -\tfrac{1}{2}\, \tilde{\boldsymbol{\omega}} \otimes q,$$

which is P-(3). *Derived here.*

Code convention. MuJoCo's free joint stores Q (body→world) in `qpos`; the runner forms $q = Q^*$ and reads $\boldsymbol{\omega}$ in body axes from `qvel`. Measurement noise (Sec. VI-E) left-multiplies a small Euler-angle quaternion into q and adds Gaussian samples to $\boldsymbol{\omega}$; the *true* q is kept separately for scoring.

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py`: 316–339 true and fed-back state, noise, reference, error

```
316             t = float(data.time)
317             q_true = quat_conj(
318                 np.asarray(data.qpos[qa + 3 : qa + 7], dtype=float).copy()
319             )
320             omega_true = np.asarray(
321                 data.qvel[va + 3 : va + 6], dtype=float
322             ).copy()
323
324             # The control arm receives corrupted feedback; the true plant state
325             # is retained independently for the primary evaluation metric.
326             q_feedback = q_true.copy()
327             omega_feedback = omega_true.copy()
328             if args.attitude_noise_std > 0.0:
329                 dq = euler_to_quat(
330                     *rng.normal(0.0, args.attitude_noise_std, size=3)
331                 )
332                 q_feedback = quat_mul(dq, q_feedback)
333             if args.rate_noise_std > 0.0:
334                 omega_feedback += rng.normal(0.0, args.rate_noise_std, size=3)
335
336             ref_euler = reference_euler(t, args.mission, args.duration)
337             q_ref = reference_quat(t, args.mission, args.duration)
338             qe_true, eq_true = quat_error(q_ref, q_true)
339             qe_feedback, eq_feedback = quat_error(q_ref, q_feedback)
```

3.3 Error quaternion, sign selection, and angle P-(4)–P-(6)

$q_e = q_d \otimes q^{-1}$. Writing $q_e = [\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\boldsymbol{n}^\mathsf{T}]^\mathsf{T}$ gives $\|\boldsymbol{q}_{e,v}\| = |\sin\frac{\theta}{2}|$ and $|q_{e,0}| = |\cos\frac{\theta}{2}|$, so $\theta_q = 2\,\mathrm{atan2}(\|\boldsymbol{q}_{e,v}\|, |q_{e,0}|) \in [0, \pi]$ is the smallest rotation angle and is invariant to $q_e \mapsto -q_e$. The selection $\boldsymbol{e}_q = \mathrm{sgn}_+(q_{e,0})\boldsymbol{q}_{e,v}$ picks the hemisphere $q_{e,0} \geq 0$; it is discontinuous at $q_{e,0} = 0$, consistent with the fact that no continuous feedback stabilizes attitude globally [3, 4]; global designs need hybrid [5] or geometric-algebra [6] methods. *Standard; the paper states the local scope.*

`src/gra_control/math3d.py`: 40–51 $q_e = q_d \otimes q^*$, sgn_+ (`ifqe[0]<0`), θ_q

```
40 def quat_error(q_ref: np.ndarray, q_meas: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
41     """Return shortest-path q_ref * conj(q_meas) and its vector part."""
42     qe = quat_normalize(quat_mul(quat_normalize(q_ref), quat_conj(quat_normalize(q_meas))))
43     eq = qe[1:].copy()
44     if qe[0] < 0:
45         eq *= -1.0
46     return qe, eq
47
48
49 def quat_angle(qe: np.ndarray) -> float:
50     qe = quat_normalize(qe)
51     return float(2.0*np.arctan2(np.linalg.norm(qe[1:]), max(abs(qe[0]), 1e-12)))
```

3.4 Error kinematics P-(7)–P-(8)

With $\dot{q}_d = -\frac{1}{2}\tilde{\boldsymbol{\omega}}_d \otimes q_d$ (same passive convention) and $\dot{(q^*)} = \frac{1}{2} q^* \otimes \tilde{\boldsymbol{\omega}}$,

$$\dot{q}_e = \dot{q}_d \otimes q^* + q_d \otimes \dot{(q^*)} = -\tfrac{1}{2}\tilde{\boldsymbol{\omega}}_d \otimes q_e + \tfrac{1}{2} q_e \otimes \tilde{\boldsymbol{\omega}}.$$

Insert $q_e \otimes q_e^* = 1$: $\tilde{\boldsymbol{\omega}}_d \otimes q_e = q_e \otimes [0, (R(q_e)^{\mathsf{T}}\boldsymbol{\omega}_d)^{\mathsf{T}}]^{\mathsf{T}}$. Hence

$$\dot{q}_e = \tfrac{1}{2} q_e \otimes [0, \boldsymbol{\omega}_e^{\mathsf{T}}]^{\mathsf{T}}, \quad \boldsymbol{\omega}_e = \boldsymbol{\omega} - R(q_e)^{\mathsf{T}}\boldsymbol{\omega}_d, \quad \dot{q}_{e,0} = -\tfrac{1}{2}\boldsymbol{q}_{e,v}^{\mathsf{T}}\boldsymbol{\omega}_e, \quad \dot{\boldsymbol{q}}_{e,v} = \tfrac{1}{2}(q_{e,0}\boldsymbol{\omega}_e + \boldsymbol{q}_{e,v} \times \boldsymbol{\omega}_e). \tag{2}$$

The first three are P-(7)–P-(8). *Derived here.*

3.5 References and the desired rate $\boldsymbol{\omega}_d$

Mission references are closed-form Euler 3-2-1 triples (Sec. VI-A) mapped to quaternions by the half-angle formula [1].

`src/gra_control/math3d.py` : 28–37 Euler 3-2-1 to quaternion

```python
def euler_to_quat(phi: float, theta: float, psi: float) -> np.ndarray:
    cr, sr = np.cos(phi/2), np.sin(phi/2)
    cp, sp = np.cos(theta/2), np.sin(theta/2)
    cy, sy = np.cos(psi/2), np.sin(psi/2)
    return quat_normalize(np.array([
        cr*cp*cy + sr*sp*sy,
        sr*cp*cy - cr*sp*sy,
        cr*sp*cy + sr*cp*sy,
        cr*cp*sy - sr*sp*cy,
    ]))
```

`src/gra_control/math3d.py` : 54–82 mission references (Roll, Full3, Hard45, Hardstep, Yaw360; Hover)

```python
def reference_euler(t: float, mission: str, duration: float) -> np.ndarray:
    if mission == "hover":
        return np.zeros(3)
    if mission == "roll":
        return np.array([0.25*np.sin(0.7*t), 0.0, 0.0])
    if mission == "full3":
        return 0.5*np.sin(t + np.array([0.0, np.pi/2, np.pi]))
    if mission == "hard45":
        return np.array([
            0.55*np.sin(0.75*t) + 0.20*np.sin(2.10*t + 0.40),
            0.45*np.sin(0.95*t + np.pi/3) + 0.18*np.sin(1.75*t),
            0.65*np.sin(0.55*t + np.pi) + 0.25*np.sin(1.35*t + 0.20),
        ])
    if mission == "hardstep":
        if t < 1.0:
            return np.zeros(3)
        phase = int((t-1.0)//2.0) % 4
        return np.array([
            [0.75, -0.45, 0.55], [-0.65, 0.55, -0.70],
            [0.50, 0.50, 0.90], [-0.80, -0.35, 0.20],
        ][phase])
    if mission == "yaw360":
        s = np.clip(t/max(duration, 1e-9), 0.0, 1.0)
        return np.array([0.0, 0.0, 2*np.pi*(3*s*s-2*s*s*s)])
    raise ValueError(f"Unknown mission: {mission}")


def reference_quat(t: float, mission: str, duration: float) -> np.ndarray:
    return euler_to_quat(*reference_euler(t, mission, duration))
```

The feedforward rate is a central difference of q_d with $h = 10^{-4}$ s. Using $q_d(t \pm h) = q_d \pm h\dot{q}_d + O(h^2)$ and $\dot{q}_d = -\frac{1}{2}\tilde{\boldsymbol{\omega}}_d \otimes q_d$,

$$q_d(t+h) \otimes q_d(t-h)^* = 1 + 2h\,\dot{q}_d \otimes q_d^* + O(h^2) = [1,\, -h\boldsymbol{\omega}_d^{\mathsf{T}}]^{\mathsf{T}} + O(h^2),$$

so $\boldsymbol{w} = \frac{2}{2h}\,\mathrm{vec}(\cdot) = -\boldsymbol{\omega}_d$, and the `left-neg` mode returns $+\boldsymbol{\omega}_d$, the desired-frame rate of P-(7)–P-(8). *Derived here.*

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 89–107 `reference_rate`: central difference, four sign/side modes

```
def reference_rate(t, mission, duration, mode, h=1e-4):
    """Reference angular rate from a central difference of reference_quat.

    'left'  uses vec(q(t+h) q(t-h)^-1); 'right' uses vec(q(t-h)^-1 q(t+h));
    the '-neg' variants flip the sign. Which one matches this codebase's
    passive convention is decided empirically by select_rate_ff.py.
    """
    t0 = max(t - h, 0.0)
    qa = reference_quat(t0, mission, duration)
    qb = reference_quat(t + h, mission, duration)
    base = mode.replace("-neg", "")
    if base == "left":
        dq = quat_mul(qb, quat_conj(qa))
    else:
        dq = quat_mul(quat_conj(qa), qb)
    if dq[0] < 0.0:
        dq = -dq
    w = 2.0 * dq[1:] / ((t + h) - t0)
    return -w if mode.endswith("-neg") else w
```

Status. Verified (`reproduce/check_rate_frame.py`): propagating q_d one step with the `left-neg` rate leaves a residual of 4×10^{-9} (Hard45), 2×10^{-9} (Full3) and 4×10^{-11} (Yaw360); the other three modes miss by 2×10^{-5}–10^{-4}. The mode was originally chosen empirically (`select_rate_ff.py`); this derivation confirms it.

Implementation note. The PD+ff law P-(15) damps $\boldsymbol{\omega} - \boldsymbol{\omega}_d$, exactly as written in the paper; it does not apply the transport $R(q_e)^\mathsf{T}$ of P-(8). The difference is bounded by $\|(I - R(q_e)^\mathsf{T})\boldsymbol{\omega}_d\| = 2|\sin\frac{\theta_q}{2}|\,\|\boldsymbol{\omega}_{d,\perp}\| \le \theta_q\|\boldsymbol{\omega}_d\|$. Measured on the published $\rho = 3$ runs it is 0.3–0.7% of $\|\boldsymbol{\omega}_d\|$ on Full3 and Hard45 and $< 10^{-4}$% on Roll and Yaw360 (single-axis references commute with q_e). On Hardstep the reference is piecewise constant, so $\boldsymbol{\omega}_d = 0$ except for one 0.5-ms control sample at each step instant, where the difference straddles the jump ($\|\boldsymbol{\omega}_d\| \approx 8.3 \times 10^3$ rad/s) and the baseline torque saturates for that sample in both arms. No reported number is affected by the notation; the Hardstep spike is part of every Hardstep result, identically in both arms.

4 Rigid body, inertia, and the torque box: P-(2), P-(9)

P-(2) is Euler's rotational equation in body axes [2]; MuJoCo integrates it [7] with the model files from MuJoCo Menagerie [8] (X2) and the 1.5× rotor design derived from its Crazyflie 2 model (CF2). P-(9) is the similarity transform of the principal-axis tensor. Controller gains use the principal moments `body_inertia` in their reported order ($\boldsymbol{j}_{\text{tune}}$, Table I).

Torque box. $\ell_i = \max(j_{\text{tune},i}\nu_i, 10^{-6})$ with $\boldsymbol{\nu} = (12, 12, 10)\,\text{rad/s}^2$: the torque that gives axis i an angular acceleration ν_i. For both rotor platforms the `model` policy returns this inertia limit.

`src/gra_control/platforms/drone.py`: 76–98 `torque_limits`

```
def torque_limits(
    platform_name: str,
    inertia: np.ndarray,
    model: mujoco.MjModel,
    policy: str,
) -> np.ndarray:
    inertia_limit = np.maximum(inertia*np.array([12.0, 12.0, 10.0]), 1e-6)
    if policy == "inertia":
        return inertia_limit
    if policy == "legacy-learned":
        return np.maximum(inertia_limit, np.array([0.03, 0.03, 0.02]))
    if policy == "model":
```

```
        if platform_name == "cf2":
            names = ["x_moment", "y_moment", "z_moment"]
            aids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, n) for n in names])
            gear = np.abs(np.array([
                model.actuator_gear[aids[0], 3], model.actuator_gear[aids[1], 4],
                model.actuator_gear[aids[2], 5],
            ]))
            ctrl = np.max(np.abs(model.actuator_ctrlrange[aids]), axis=1)
            return np.minimum(inertia_limit, gear*ctrl)
        return inertia_limit
    raise ValueError(f"Unknown torque policy: {policy}")
```

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py`:236–254 inertia, limits and the frozen $\rho = 3$ baseline

```
def run(args: argparse.Namespace) -> dict[str, Any]:
    spec = SPECS[args.platform]
    model = mujoco.MjModel.from_xml_path(str(spec.model))
    model.opt.timestep = args.dt
    data = mujoco.MjData(model)
    _, body_id, qa, va = _free_joint(model)
    inertia = model.body_inertia[body_id].copy()
    mass = float(np.sum(model.body_mass))
    gravity = abs(float(model.opt.gravity[2]))
    limits = torque_limits(args.platform, inertia, model, args.torque_policy)

    # These fixed attributes make AdvancedBaseline exactly the archived tuned
    # PD+desired-rate-feedforward controller at rho=3.
    args.baseline = BASELINE_FAMILY
    args.bw_scale = BASELINE_RHO
    args.rate_ff = BASELINE_RATE_FF
    args.no_model_ff = not BASELINE_MODEL_FF
    args.baseline_rate_hz = 0.0
    baseline = AdvancedBaseline(args, inertia, limits, BASE_WN_RAD_S)
```

5 Allocation and the height loop: P-(10)–P-(12)

5.1 Allocation matrix P-(10)

A rotor at $\boldsymbol{p}_i$ with thrust $[0, 0, f_i]^\mathsf{T}$ gives torque $\boldsymbol{p}_i \times [0, 0, f_i]^\mathsf{T} = [p_{i,y} f_i,\ -p_{i,x} f_i,\ 0]^\mathsf{T}$; the drag reaction adds $\kappa_i f_i$ about z [9]. Summing, $[T, \boldsymbol{\tau}^\mathsf{T}]^\mathsf{T} = B\boldsymbol{f}$ with columns $(1,\ p_{i,y},\ -p_{i,x},\ \kappa_i)$. For four rotors B is square and invertible, so $B^{-1} = B^\dagger$. *Derived here.*

`src/gra_control/platforms/drone.py`:67–73 B from rotor-site positions and actuator gear; `np.linalg.inv(B)`

```
def _mixer(model: mujoco.MjModel) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    names = [f"thrust{i}" for i in range(1, 5)]
    aids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_ACTUATOR, n) for n in names])
    sids = np.array([_name_id(model, mujoco.mjtObj.mjOBJ_SITE, n) for n in names])
    pos = model.site_pos[sids]
    B = np.vstack([np.ones(4), pos[:, 1], -pos[:, 0], model.actuator_gear[aids, 5]])
    return aids, B, np.linalg.inv(B), model.actuator_ctrlrange[aids].copy()
```

5.2 Clipped inverse allocation P-(11) and the CF2 rotor limit

$\boldsymbol{f}_{\text{req}} = B^{-1}[T, \boldsymbol{u}^\mathsf{T}]^\mathsf{T}$ is clipped to $[\mathbf{0}, \boldsymbol{f}_{\max}]$; the realized wrench is $B\boldsymbol{f}_{\text{act}}$. This is not an optimization-based allocator [10]. The CF2 authority scale multiplies only the upper force limit ($0.0875 \rightarrow 0.13125\,\text{N}$, Table I).

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` : 280–285 `--actuator-scale` (1.5 for CF2, 1.0 for X2)

```
    # Both paper platforms use four explicitly allocated rotor forces.
    act_ids, mixer, mixer_inv, ctrl_range = _mixer(model)
    if args.actuator_scale <= 0.0:
        raise ValueError("--actuator-scale must be positive")
    ctrl_range[:, 1] *= args.actuator_scale
    model.actuator_ctrlrange[act_ids, 1] = ctrl_range[:, 1]
```

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` : 412–422 $\boldsymbol{f}_{\text{req}}$, clip, rotor-saturation flag, realized wrench

```
            actuator_requested = mixer_inv @ np.r_[collective_requested, tau_command]
            actuator_command = np.clip(
                actuator_requested, ctrl_range[:, 0], ctrl_range[:, 1]
            )
            rotor_sat_mask = (
                np.isclose(actuator_command, ctrl_range[:, 0], atol=1e-7)
                | np.isclose(actuator_command, ctrl_range[:, 1], atol=1e-7)
            )
            realized_wrench = mixer @ actuator_command
            collective_realized = float(realized_wrench[0])
            tau_realized = realized_wrench[1:4].copy()
```

5.3 Collective thrust P-(12)

Vertical balance $T\,R(Q)_{33} - Mg = M\ddot{z}$ with the imposed $\ddot{z} = 6(z_d - z) - 4\dot{z}$ gives $T = M[g + 6(z_d - z) - 4\dot{z}]/R_{33}$: a PD altitude loop with $\omega_{n,z} = \sqrt{6} = 2.45\,\text{rad/s}$ and $\zeta_z = 4/(2\sqrt{6}) = 0.82$. The guard $R_{33} \geq 0.35$ caps tilt compensation at $\arccos 0.35 = 69.5°$ (factor ≤ 2.86), and $T \leq 2.5Mg$. *Derived here.*

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` : 397–410 collective thrust

```
            rotation = np.asarray(data.xmat[body_id], dtype=float).reshape(3, 3)
            z = float(data.qpos[qa + 2])
            vz = float(data.qvel[va + 2])
            tilt = max(float(rotation[2, 2]), 0.35)
            collective_requested = mass * (
                gravity + 6.0 * (1.0 - z) - 4.0 * vz
            ) / tilt
            collective_requested = float(
                np.clip(
                    collective_requested,
                    0.0,
                    2.5 * mass * max(gravity, 1.0),
                )
            )
```

6 Classical baselines: P-(13)–P-(15)

6.1 Gain design P-(13)

For one principal axis with small error, $e_q \approx \frac{\theta}{2}$ and, from (2) with $q_{e,0} \approx 1$, $\dot{e}_q \approx \frac{1}{2}\omega$. With $j\dot{\omega} = -k_q e_q - k_\omega \omega$ and $\omega = \dot{\theta}$,

$$j\ddot{\theta} + k_\omega \dot{\theta} + \tfrac{k_q}{2}\theta = 0 \quad \Longrightarrow \quad \omega_n^2 = \frac{k_q}{2j}, \qquad 2\zeta\omega_n = \frac{k_\omega}{j},$$

so $k_q = 2j\omega_n^2$ and $k_\omega = 2\zeta j \omega_n$; the factor 2 compensates the half angle. Because the X2 tensor is not diagonal and $\boldsymbol{j}_{\text{tune}} \neq \text{diag}(\boldsymbol{J})$, ω_n is a tuning label, not an exact closed-loop frequency. *Derived here.*

6.2 Fixed PD P-(14) (Table II, lowest blocks)

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 163–165 $\boldsymbol{\omega}_n = (4, 4, 3.5)$, $\boldsymbol{k}_q$, $\boldsymbol{k}_\omega$ with $\zeta = 0.95$

```
    wn = np.array([4.0, 4.0, 3.5])
    kq = 2.0 * inertia * wn * wn
    kw = 2.0 * 0.95 * wn * inertia
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef.py`: 226–230 P-(14) and the PES target P-(24)

```
            ref_euler = reference_euler(t, args.mission, args.duration)
            qref = reference_quat(t, args.mission, args.duration)
            qe, eq = quat_error(qref, q_meas)
            tau0 = np.clip(-kq * eq - kw * omega, -limits, limits)
            target = np.clip(-args.k_error * eq - args.k_rate * omega, -1.0, 1.0)
```

6.3 PD with desired-rate feedforward P-(15) and the ρ scaling

The $\rho = 1$ ladder (Table II) used the archived runner's PD+ff branch, with $\boldsymbol{\omega}_d$ from Sec. 3.5:

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 463–470 P-(15) (integral gain zero in the paper's runs)

```
            else:
                if args.rate_ff == 'none':
                    omega_d = np.zeros(3)
                else:
                    omega_d = reference_rate(t, args.mission, args.duration,
                                             args.rate_ff)
                tau0_unsat = -kq * eq - kw * (omega - omega_d) - ki * eq_integral
                tau0 = np.clip(tau0_unsat, -limits, limits)
```

The $\rho = 3$ primary baseline is `AdvancedBaseline` with the backstepping family, bandwidth $\rho\boldsymbol{\omega}_n$ and model feedforward off. Its law is $\boldsymbol{\tau} = \mathrm{sat}_{\boldsymbol{\ell}}\left(\boldsymbol{J}[\dot{\boldsymbol{\omega}}_d - k_1(2\boldsymbol{e}_q) - k_2(\boldsymbol{\omega} - \boldsymbol{\omega}_d)] + \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}\right)$ with $k_1 = (\rho\omega_n)^2$, $k_2 = 2\zeta\rho\omega_n$. With model feedforward off, $\dot{\boldsymbol{\omega}}_d = 0$ and the gyroscopic term is dropped, and since `self.J` holds $\boldsymbol{j}_{\mathrm{tune}}$,

$$\boldsymbol{\tau} = \mathrm{sat}_{\boldsymbol{\ell}}\left(-2\boldsymbol{j}_{\mathrm{tune}} \odot (\rho\boldsymbol{\omega}_n)^2 \odot \boldsymbol{e}_q - 2\zeta\, \boldsymbol{j}_{\mathrm{tune}} \odot \rho\boldsymbol{\omega}_n \odot (\boldsymbol{\omega} - \boldsymbol{\omega}_d)\right),$$

which is P-(13)–P-(15) with $\boldsymbol{\omega}_n \mapsto \rho\boldsymbol{\omega}_n$. *Derived here; verified bit-exact in 20/20 cells (Table 2).*

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py`: 78–85 frozen baseline: $\rho = 3$, $\zeta = .95$, backstepping, `left-neg`, no model feedforward

```
# Frozen paper-facing baseline.  AdvancedBaseline with backstepping and
# --no-model-ff is algebraically PD + desired-rate feedforward.
BASE_WN_RAD_S = np.array([4.0, 4.0, 3.5])
BASELINE_RHO = 3.0
BASELINE_ZETA = 0.95
BASELINE_FAMILY = "backstepping"
BASELINE_RATE_FF = "left-neg"
BASELINE_MODEL_FF = False
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 138–141 backstepping gains $k_1 = \omega^2$, $k_2 = 2\zeta\omega$ at $\omega = \rho\omega_n$

```
        elif b in ("backstepping", "nmpc"):
            # Computed-torque backstepping (aero2 form): desired accel
            # -k1 e - k2 w_e, then inverse dynamics. NMPC uses it as nominal.
            self.k1, self.k2 = self.wn ** 2, 2.0 * self.zeta * self.wn
```

`experiments/nengo_nef_replication_2026-08-29/run_mujoco_nef_baselines.py`: 217–245 `AdvancedBaseline.torque`: all six families behind one interface

```
217     def torque(self, t, eq, omega, q_meas):
218         if self.hold is not None and self.count % self.every != 0:
219             self.count += 1
220             return self.hold
221         self.count += 1
222         # NMPC self-check: compare last update's one-period-ahead prediction
223         # with where the attitude actually is now. A convention error in the
224         # propagation shows up as prediction error of the size of the motion.
225         if self.pred is not None:
226             self.pred_err.append(quat_angle(quat_error(self.pred, q_meas)[0]))
227             self.hold_err.append(quat_angle(quat_error(self.q_prev, q_meas)[0]))
228         wd, wd_dot = self._wd(t), self._wd_dot(t)
229         e, we = 2.0 * eq, omega - wd
230         gyro = np.cross(omega, self.J * omega) if self.model_ff else np.zeros(3)
231         b = self.a.baseline
232         if b in ("lqr", "backstepping"):
233             acc = wd_dot - self.k1 * e - self.k2 * we
234         elif b == "smc":
235             s_ = we + self.lam * e
236             acc = wd_dot - self.lam * we - self.eta * np.clip(s_ / self.phi, -1.0, 1.0)
237         elif b == "mpc":
238             acc = wd_dot - (self.K0[:, 0] * e + self.K0[:, 1] * we)
239         elif b == "nmpc":
240             acc = wd_dot - self.k1 * e - self.k2 * we
241         tau = np.clip(self.J * acc + gyro, -self.lim, self.lim)
242         if b == "nmpc":
243             tau = self._nmpc(t, q_meas, omega, tau)
244         self.hold = tau
245         return tau
```

6.4 Other control families (Sec. VI-B, portability)

The same `torque` method implements LQR (per-axis double integrator, $K = [\omega^2, \sqrt{2}\,\omega]$), sliding mode ($s = \boldsymbol{\omega}_e + \lambda \boldsymbol{e}$ with a boundary layer), unconstrained finite-horizon MPC (first-move gain), and NMPC (grid receding horizon around the backstepping law); PID+ff adds conditional-integration anti-windup at rotor level (`run_controllers_gated.py`). The residual only replaces $\boldsymbol{u}_0$ in P-(27); Proposition 1 holds for any $\boldsymbol{u}_{0,k} \in \mathcal{U}$, and every family's output is in $\mathcal{U}$: LQR, backstepping, SMC and MPC are saturated in line 241 above, NMPC clips every candidate torque inside `_nmpc`, and the PID path clips at line 353 of `run_controllers_gated.py`.

7 The CMG unit: P-(16)–P-(22)

7.1 Paired-current encoding P-(16)

$\bar{\boldsymbol{u}} = \boldsymbol{u}_0 \oslash \boldsymbol{\ell} \in [-1, 1]^m$ because $\boldsymbol{u}_0$ is already saturated, and $\boldsymbol{I}^{\pm} = I_b \mathbf{1} + I_g \max(\pm\bar{\boldsymbol{u}}, 0) \in [I_b, I_b + I_g]^m$. In each pair at most one current exceeds I_b: one sign carries the request, the other rests.

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`: 318–334 normalized request, paired currents, clipped PES target

```
318         u0 = np.clip(np.asarray(baseline, dtype=float), -self.limits, self.limits)
319         normalized = np.divide(
320             u0,
321             self.limits,
322             out=np.zeros_like(u0),
323             where=self.limits > 1e-12,
324         )
325         self.external.current[:] = np.concatenate(
326             [
327                 self.current_bias + self.current_gain * np.maximum(normalized, 0.0),
```

```
                self.current_bias + self.current_gain * np.maximum(-normalized, 0.0),
            ]
        )
        self.external.target[:] = np.clip(
            np.asarray(learning_target, dtype=float), -1.0, 1.0
        )
        self._last_target[:] = self.external.target
```

7.2 Continuous dynamics P-(17)–P-(19) and the membrane step P-(22)

The unit is conductance-based in the Hodgkin–Huxley sense [11]: a gate r_j relaxes to a logistic target set by the membrane, and a gate-dependent shunt pulls the membrane toward a reversal level. It emits no spikes and has no reset. The membrane equation is affine in v,

$$\dot{v} = -\underbrace{\left(\tfrac{1}{\tau} + c\right)}_{\lambda} v + \underbrace{\tfrac{v_{\text{rest}}+I}{\tau} + c\, v_{\text{reset}}}_{d},$$

so with λ, d frozen over $[t_k, t_k + h]$ the integrating factor gives $v(t_k + h) = e^{-\lambda h} v_k + (1 - e^{-\lambda h})\, d/\lambda$, which is P-(22), an exponential integrator [12]. Since $0 < e^{-\lambda h} < 1$, the step is a convex combination of v_k and the frozen equilibrium d/λ. *Derived here.*

7.3 Gate step P-(20)–P-(21)

The implemented step is forward Euler with a clip: $r^+ = \text{clip}_{[0,1]}\left[(1 - \tfrac{h}{\tau_r})r + \tfrac{h}{\tau_r} b\right]$. For $0 < h \le \tau_r$ the weights are nonnegative and sum to one, so with $r, b \in [0, 1]$ the clip is inactive. The exponential-Euler alternative $r^+ = e^{-h/\tau_r} r + (1 - e^{-h/\tau_r}) b$ is convex for every h; at $h/\tau_r = 0.1$ its weights are 0.905/0.095 against 0.9/0.1, and all results use the implemented step. The code updates r first and uses the updated r^+ in c_j, as the paper states. The logistic argument is written $\alpha_h(v - (v_{\text{th}} - \gamma r))$, identical to P-(20).

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`: 71–89 Nengo `NeuronType.step`: gate (Euler + clip), membrane P-(22), activity clip(Gr)

```
    def step(self, dt, J, output, voltage, gate):
        x = np.clip(
            self.alpha * (voltage - (self.v_threshold - self.gamma * gate)),
            -60.0,
            60.0,
        )
        gate_target = 1.0 / (1.0 + np.exp(-x))
        gate += dt * (gate_target - gate) / self.tau_r
        np.clip(gate, 0.0, 1.0, out=gate)

        gain = self.base_gain
        if self.adaptive:
            gain = gain + self.gain_scaling * np.abs(J)
        conductance = gain * gate * gate
        lam = 1.0 / self.tau + conductance
        drive = self.v_rest / self.tau + J / self.tau + conductance * self.v_reset
        decay = np.exp(-lam * dt)
        voltage[:] = voltage * decay + (drive / np.maximum(lam, 1e-12)) * (1.0 - decay)
        output[:] = np.clip(self.gate_gain * gate, 0.0, 1.0)
```

`src/gra_control/gate.py` : 41–55 the direct NumPy bank: the same update, line for line

```python
    def step(self, current: np.ndarray, dt: float) -> np.ndarray:
        p = self.cfg
        current = np.asarray(current, dtype=float)
        x = np.clip(p.alpha*(self.v-(p.v_threshold-p.gamma*self.r)), -60.0, 60.0)
        target = 1.0/(1.0+np.exp(-x))
        self.r += dt*(target-self.r)/p.tau_r
        self.r[:] = np.clip(self.r, 0.0, 1.0)

        gain = p.base_gain + (p.gain_scaling*np.abs(current) if self.adaptive else 0.0)
        conductance = gain*self.r*self.r
        lam = 1.0/p.tau + conductance
        drive = p.v_rest/p.tau + current/p.tau + conductance*p.v_reset
        decay = np.exp(-lam*dt)
        self.v[:] = self.v*decay + (drive/np.maximum(lam, 1e-12))*(1.0-decay)
        return self.r.copy()
```

7.4 Bounded states (Proposition 1(i))

Gate: as above. *Membrane:* $I_j \in [I_b, I_b + I_g]$ and $c_j \geq 0$ give $d/\lambda = \frac{1/\tau}{1/\tau+c}(v_{\text{rest}} + I) + \frac{c}{1/\tau+c}v_{\text{reset}}$, a convex combination of $v_{\text{rest}} + I$ and v_{reset}; the step is a convex combination of v_k and d/λ; induction closes the bound.

7.5 Rest point, resting activity, latching, and χ (Sec. VI-C)

Rest. With $\bar{u} = 0$, $I = I_b = 0.75$ and $r \approx 0$: $v^\star = v_{\text{rest}} + I_b = 0.75$ and $b = \sigma(\alpha_h(v^\star - v_{\text{th}})) = \sigma(-12.5) = 3.7267 \times 10^{-6} \approx r^\star$ (the γr term changes the argument by $O(10^{-4})$). Resting activity $a^\star = \min(1, Gr^\star) = 0.3727$.

Latch. If $r_k \geq \frac{1}{2}$ and $v_k \geq 0$ (Prop. 1(i) gives $v \geq 0$ since $v_{\text{reset}} = v_{\text{rest}} = 0$), then $\alpha_h(v - v_{\text{th}} + \gamma r) \geq \alpha_h(0 - 1 + 1) = 0$, so $b_k \geq \frac{1}{2}$ and the convex step keeps $r_{k+1} \geq \frac{1}{2}$: an opened gate cannot close.

Why χ is inert. At rest $c_j = g_0 r_j^2 = 120\,(3.73 \times 10^{-6})^2 = 1.7 \times 10^{-9}\,\text{s}^{-1}$, against a leak $\tau^{-1} = 20\,\text{s}^{-1}$. After latching ($r = 1$, $I = I_b$): $c = 120$ ($\chi = 0$) gives $v^\star = 15/140 = 0.107$, and $c = 142.5$ ($\chi = 1$) gives $v^\star = 15/162.5 = 0.092$, below v_{th} in both cases, while the latched activity stays at $a = 1$. The readout never sees the difference. *Derived here; these are the paper's numbers in Sec. VI-C.*

8 Readout, PES, residual, and the gate: P-(23)–P-(28)

8.1 Readout P-(23)

$\boldsymbol{a} = \text{clip}_{[0,1]}(G\boldsymbol{r}^+)$ saturates once $r \geq 1/G = 10^{-5}$. $W_0 = [I_m, -I_m]$ gives $\boldsymbol{s} = \text{clip}(\boldsymbol{a}^+ - \boldsymbol{a}^-)$: pair subtraction recovers a signed quantity from nonnegative features.

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py` : 143–150 $W_0 = [I, -I]$ and the Nengo learning rate

```python
        self.W0 = np.concatenate(
            [np.eye(self.channels), -np.eye(self.channels)], axis=1
        )

        # Nengo's SimPES applies -eta_N * dt/n * e a^T.  This conversion makes
        # that increment equal to the frozen controller's
        # +eta_direct * (target-decoded) a^T.
        self.nengo_learning_rate = self.learning_rate * self.n_units / self.dt
```

8.2 PES update P-(24)–P-(26)

PES [13, 14] is the local gradient step on $E = \frac{1}{2}\|\boldsymbol{y}^* - W\boldsymbol{a}\|^2$: $-\partial E/\partial W = \boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$. The code uses the clipped readout inside $\boldsymbol{\varepsilon}$. The leak $-\lambda_W W$ is the σ-modification of robust adaptive control [15, 16]; $\Pi_{\mathcal{W}}$ clips each weight to $[-6, 6]$. The residual uses W_k *before* the update. *Standard.*

Nengo conversion. Nengo 4.1.0's `SimPES` applies $\Delta W = -(\kappa\, h/n)\, \boldsymbol{e}\, \boldsymbol{a}^\mathsf{T}$ with n presynaptic units and error $\boldsymbol{e} = \boldsymbol{s} - \boldsymbol{y}^* = -\boldsymbol{\varepsilon}$ [17]. Hence $\Delta W = (\kappa h/n)\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$, and matching $\eta\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$ needs $\kappa = \eta n/h = 8 \times 10^{-4} \cdot 6/(5 \times 10^{-4}) = 9.6$. The leak is applied to the pre-step weights and the projection after the simulator step. The teaching graph adds a one-step latency; the resulting difference from the direct controller is the 2.013×10^{-6} of Table 2.

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`: 191–214 PES connection; error = clipped decoded − target

```
                self.decoder = nengo.Connection(
                    self.population.neurons,
                    decoded,
                    transform=self.W0,
                    synapse=None,
                    learning_rule_type=nengo.PES(
                        learning_rate=self.nengo_learning_rate,
                        pre_synapse=None,
                    ),
                )
            nengo.Connection(decoded, decoded_clipped, synapse=None)
            error = nengo.Node(
                lambda t, x: x[: self.channels] - x[self.channels :],
                size_in=2 * self.channels,
                size_out=self.channels,
                label="PES error = decoded - target",
            )
            self.error_node = error
            # The frozen direct controller computes PES from clip(Wa,-1,1).
            # Keeping the raw and clipped signals separate makes saturation
            # explicit and preserves exact controller semantics.
            nengo.Connection(decoded_clipped, error[: self.channels], synapse=None)
            nengo.Connection(target, error[self.channels :], synapse=None)
            nengo.Connection(error, self.decoder.learning_rule, synapse=None)
```

`experiments/nengo_nef_replication_2026-08-29/nengo_cmg.py`: 335–352 leak on pre-step weights, projection to $[-6, 6]$, residual P-(27)

```
        weights_before = self._weights.copy()
        self.sim.step()

        self._last_decoded[:] = np.asarray(self._decoded, dtype=float)
        self._last_error[:] = np.asarray(self._error, dtype=float)

        # Preserve the frozen controller's decoder regularization.  PES itself
        # is Nengo's built-in learning operator; only leak and safety clipping
        # are imposed here after its update.
        # Direct semantics are W_new = W_old + PES - leak*W_old.  Subtracting
        # from the pre-step value avoids scaling the just-computed PES update.
        self._weights[:] -= self.leak * weights_before
        np.clip(self._weights, -6.0, 6.0, out=self._weights)

        signed = np.clip(self._last_decoded, -1.0, 1.0)
        residual_requested = self.beta * signed * self.limits
        command = np.clip(u0 + residual_requested, -self.limits, self.limits)
        residual = command - u0
```

src/gra_control/gate.py : 110–134 the direct controller: P-(16), P-(23), P-(26), P-(27)

```
        u0 = np.clip(np.asarray(baseline, dtype=float), -self.limits, self.limits)
        normalized = np.divide(u0, self.limits, out=np.zeros_like(u0), where=self.limits > 1e-12)
        current = np.concatenate([
            self.cfg.current_bias + self.cfg.current_gain*np.maximum(normalized, 0.0),
            self.cfg.current_bias + self.cfg.current_gain*np.maximum(-normalized, 0.0),
        ])
        gate = self.units.step(current, dt)
        activity = np.clip(self.cfg.gate_gain*gate, 0.0, 1.0)
        signed = np.clip(self.W@activity, -1.0, 1.0)

        if self.mode == "online_pes" and learning_target is not None:
            target = np.clip(np.asarray(learning_target, dtype=float), -1.0, 1.0)
            self.W += self.learning_rate*np.outer(target-signed, activity) - self.leak*self.W
            self.W[:] = np.clip(self.W, -6.0, 6.0)

        neural = signed*self.limits
        if self.mode == "blend":
            command = (1.0-self.beta)*u0 + self.beta*neural
            residual = command-u0
        else:
            residual = self.beta*neural
            command = u0+residual
        command = np.clip(command, -self.limits, self.limits)
        residual = command-u0
        return AugmentorOutput(command, u0, residual, activity, gate)
```

8.3 Bounded residual P-(27) and the saturation-aware gate P-(28)

At $|u_{0,i}| = \ell_i$ a residual component of the same sign is clipped away, and an opposing one removes up to $\beta\ell_i$ of authority exactly when the loop needs all of it. The gate zeroes such opposing components and leaves PES untouched.

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 370–378 requested residual $\beta\boldsymbol{\ell} \odot \boldsymbol{s}$ and its consistency check

```
                # Requested residual before the outer torque box.  The archived
                # augmentor returns only the post-clip residual, so this explicit
                # reconstruction is essential to audit gating and clipping.
                raw_residual = args.beta * limits * decoded_clipped
                reconstructed = np.clip(tau0 + raw_residual, -limits, limits)
                max_augmentor_consistency_error = max(
                    max_augmentor_consistency_error,
                    float(np.max(np.abs(reconstructed - native_out.command))),
                )
```

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 380–395 gate P-(28), $\boldsymbol{u}_k = \mathrm{sat}_{\boldsymbol{\ell}}(\boldsymbol{u}_0 + \beta\boldsymbol{\ell} \odot \tilde{\boldsymbol{s}})$, realized residual

```
            baseline_loaded = np.abs(tau0) >= loaded_threshold
            opposing = baseline_loaded & (raw_residual * tau0 < 0.0)
            output_gate_mask = np.ones(3, dtype=float)
            if args.output_gate in ("torque-saturation-opposition", FREEZE_MODE):
                output_gate_mask[opposing] = 0.0
            if args.output_gate == FREEZE_MODE and augmentor is not None and np.any(opposing):
                # Anti-windup analogue: no decoder movement on an axis whose residual is blocked.
                # Neuron and gate states are untouched; only the pre-step weight rows are restored.
                augmentor._weights[opposing, :] = weights_before[opposing, :]
                weights = augmentor.W

            gated_residual = output_gate_mask * raw_residual
            tau_requested = tau0 + gated_residual
            tau_command = np.clip(tau_requested, -limits, limits)
            realized_residual = tau_command - tau0
            torque_box_sat = np.abs(tau_requested) >= limits - 1e-12
```

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 309–312 load threshold

```
    gate_epsilon = args.gate_epsilon_fraction * limits + args.gate_epsilon_nm
    loaded_threshold = np.maximum(
        args.gate_load_threshold * limits - gate_epsilon, 0.0
    )
```

Implementation note. The code tests "loaded" as $|u_{0,i}| \geq \ell_i - 10^{-12}\,\mathrm{N\,m}$ (threshold factor 1.0, absolute tolerance 10^{-12}), a floating-point guard for "$|u_{0,i}| \geq \ell_i$" in P-(28), since the saturated baseline equals ℓ_i only to rounding. The gate masks the *output* only; the Nengo augmentor's PES update never sees the mask. The mode `torque-saturation-opposition-freeze-pes` (decoder rows frozen on gated axes) was part of the exploratory twelve-configuration screen of Sec. VI-F only.

8.4 Proposition 1, full proof

(i) Sec. 7. *(ii)* $\Pi_{\mathcal{W}}$ maps into $\mathcal{W}$. *(iii)* $\boldsymbol{u}_0 \in \mathcal{U}$, and the outer saturation gives $\boldsymbol{u} \in \mathcal{U}$. Componentwise sat_{ℓ_i} is 1-Lipschitz and fixes $u_{0,i}$, so $|\delta_i| = |\,\mathrm{sat}_{\ell_i}(u_{0,i} + \beta\ell_i\tilde{s}_i) - \mathrm{sat}_{\ell_i}(u_{0,i})| \leq \beta\ell_i|\tilde{s}_i| \leq \beta\ell_i|s_i| \leq \beta\ell_i$, using $|\tilde{s}_i| \leq |s_i| \leq 1$ from P-(28). With $\beta = 0$, $\boldsymbol{u} = \mathrm{sat}(\boldsymbol{u}_0) = \boldsymbol{u}_0$, and the common allocator returns the same rotor forces at the same state and collective. □

Status. Verified: exact recovery at $\beta = 0$ holds with relative difference 0 for four new baseline families (Table 2).

9 Proposition 2 and the latched regime: P-(29)–P-(30)

9.1 Proposition 2, proof, and the clip lemma

Fix $\boldsymbol{a} \neq 0$ and $\boldsymbol{y}^*$, take $\lambda_W = 0$ with clip and projection inactive, and let $\boldsymbol{z}_k = W_k\boldsymbol{a}$. Then $\boldsymbol{z}_{k+1} = \boldsymbol{z}_k + \eta\|\boldsymbol{a}\|^2\boldsymbol{\varepsilon}_k$ and $\boldsymbol{\varepsilon}_{k+1} = (1 - \eta\|\boldsymbol{a}\|^2)\boldsymbol{\varepsilon}_k$, which is P-(29). □

Lemma 1 (The clip never enlarges the PES error)**.** *If* $\boldsymbol{y}^* \in [-1, 1]^m$ *then* $\|\,\mathrm{clip}(\boldsymbol{z}) - \boldsymbol{y}^*\| \leq \|\boldsymbol{z} - \boldsymbol{y}^*\|$ *for all* $\boldsymbol{z}$*.*

Proof. Projection onto a closed convex set is nonexpansive and $\boldsymbol{y}^*$ is a fixed point of it [18]. □

9.2 Constant-basis recursion P-(30)

With $\boldsymbol{a}_k = \bar{\boldsymbol{a}}$ and projection inactive, $\boldsymbol{z}_k = W_k\bar{\boldsymbol{a}}$ satisfies $\boldsymbol{z}_{k+1} = (1 - \lambda_W)\boldsymbol{z}_k + \eta\|\bar{\boldsymbol{a}}\|^2[\boldsymbol{y}_k^* - \mathrm{clip}(\boldsymbol{z}_k)]$. *Derived here.* When all six gates latch, $\bar{\boldsymbol{a}} = \mathbf{1}$ and $\|\bar{\boldsymbol{a}}\|^2 = 2m = 6$.

9.3 Pole, corner frequency, DC gain

With the clip inactive the recursion is linear with pole $p = 1 - \lambda_W - \eta\|\bar{\boldsymbol{a}}\|^2 = 1 - 10^{-6} - 4.8 \times 10^{-3} = 0.99520$, so

$$\tau_f = \frac{h}{1-p} = 0.1041\ \mathrm{s}, \qquad \omega_c = \frac{1}{\tau_f} = 9.60\ \mathrm{rad/s}, \qquad \text{DC gain} = \frac{\eta\|\bar{\boldsymbol{a}}\|^2}{\eta\|\bar{\boldsymbol{a}}\|^2 + \lambda_W} = 0.99979.$$

The residual therefore converges to $\beta\ell_i y_i^* = -\beta\ell_i k_e e_{q,i}$: lagged *proportional* action, not integral action (an integrator has its pole at 1 and infinite DC gain). At the $\rho = 3$ baseline bandwidth ($3{\times}4 = 12\,\mathrm{rad/s}$) the loop runs above this corner, which is the Hardstep lag of Sec. VI-F.

`experiments/theory_certificate_2026-09-12/compute_local_certificate.py`: 148–161 pole, time constant, corner, DC gain, lag, stiffness ratio

```
148
149     # latched-PES filter and graded-tail regime [eq:constantbasis]
150     a_rest = min(1.0, G_READOUT / (1 + np.exp(-ALPHA_H * (I_B - V_TH))))
151     regimes = {}
152     for name, a2 in (("latched", 6.0), ("tail", 6 * a_rest**2)):
153         pole = 1 - LAMBDA_W - ETA * a2
154         tau_f = H_STEP / (1 - pole)
155         regimes[name] = dict(a_norm2=a2, pole=pole, tau_s=tau_f, corner_rad_s=1 / tau_f,
156                              dc_gain=ETA * a2 / (ETA * a2 + LAMBDA_W),
157                              leak_mismatch=LAMBDA_W / (ETA * a2))
158     lat = regimes["latched"]
159     lag = {m: dict(gain=1 / np.sqrt(1 + (w * lat["tau_s"])**2),
160                    phase_deg=float(np.degrees(np.arctan(w * lat["tau_s"]))))
161            for m, w in MISSION_MAX_FREQ.items()}
```

Added stiffness. While $|k_e e_{q,i}| < 1$, the latched residual adds $\beta k_e \ell_i = \beta k_e j_i \nu_i$ to the PD stiffness $2j_i(\rho\omega_{n,i})^2$, a vehicle-independent ratio $\beta k_e \nu_i / (2\rho^2 \omega_{n,i}^2)$: 1.50 (roll, pitch) and 1.63 (yaw) on the fixed PD with $\beta = .5$; 0.167 and 0.181 on X2 at $\rho = 3$; 0.067 and 0.073 on CF2 at $\rho = 3$ ($\beta = .2$). The residual is a large correction on the weak baseline and a small one on the tuned baseline, consistent with Table II. *Derived here.*

Reference content. Roll, Full3 and Hard45 are sums of constant-amplitude, constant-frequency components whose instantaneous frequencies are the component frequencies [19]; the highest is 2.1 rad/s (Hard45), where the filter gain is 0.977 and the lag 12.3°. Yaw360's rate peaks at $2\pi \cdot 1.5/45 = 0.21$ rad/s.

σ-modification fixed point. At a fixed point with constant $\bar{\boldsymbol{a}}$, $\eta\|\bar{\boldsymbol{a}}\|^2\boldsymbol{\varepsilon} = \lambda_W \boldsymbol{s}$, so $\|\boldsymbol{\varepsilon}\| = \frac{\lambda_W}{\eta\|\bar{\boldsymbol{a}}\|^2}\|\boldsymbol{s}\| = 2.1 \times 10^{-4}\|\boldsymbol{s}\|$ when latched: an equality for the PES mismatch, not a bound on θ_q.

10 Neuron-basis comparison (Sec. VI-D)

10.1 Equal learning rates across neuron families

For a conventional family (LIF, ALIF, LIFRate, Izhikevich [20]) the learned connection's presynaptic signal is the filtered native output $\boldsymbol{p} = S\boldsymbol{a}$, with S the calibrated feature scale, and the class reports $W = S\Theta$ for the Nengo weights Θ. With `SimPES`, $\Delta\Theta = (\kappa h/n)\boldsymbol{\varepsilon}\boldsymbol{p}^\mathsf{T}$, hence

$$\Delta W = S\,\Delta\Theta = \frac{\kappa h S^2}{n}\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T} \quad \Longrightarrow \quad \kappa = \frac{\eta\, n}{h\, S^2}$$

for the same normalized update $\eta\boldsymbol{\varepsilon}\boldsymbol{a}^\mathsf{T}$ as CMG ($S = 1$ recovers $\kappa = 9.6$). The archived family class used $\eta n/(hS)$, an effective rate $S\eta$ ($\approx 43\times$ for Izhikevich, $S = 42.6$). The corrected runs divide the requested rate by S before construction; the archived file is not edited. *Derived here from Nengo's source*; certified in `experiments/final_robustness_gating_2026-09-20/REPORT_NEURON_BASIS_CORRECTION_2026-09-24.md`.

Implementation note. Every Izhikevich row of Table II and every number in the Sec. VI-D paragraphs (Izhikevich, LIFRate, static paired, constant basis, occupancy 3.2–4.4%) uses the corrected rate. Two statements still rest on the archived fixed-PD neuron-family campaign, which ran LIF/ALIF/LIFRate with the uncorrected rate: the Table II footnote "LIF/ALIF/LIFRate stay within 0.1% of fixed PD outside Hardstep" and the qualitative "full matrix" remarks opening Sec. VI-D. No corrected-rate fixed-PD runs exist for those three families, so these statements are not re-verified here.

experiments/nengo_nef_replication_2026-08-29/nengo_neuron_family.py : 88–93 archived rate $\eta n/(hS)$

```
        # Conventional neuron outputs are rates/spikes in Hz. Folding the
        # calibration scale into W and eta makes the learned normalized decoder
        # obey Delta W_norm = eta * (target-y) * a_norm^T.
        self.nengo_learning_rate = (
            self.learning_rate * self.n_units / (self.dt * self.feature_scale)
        )
```

experiments/final_robustness_gating_2026-09-20/izh_adapter_corrected.py : 32–40 corrected: requested rate divided by S

```
    def __init__(self, limits: np.ndarray, dt: float, beta: float, learning_rate: float, seed: int):
        calibration = json.loads((UPSTREAM / "feature_calibration.json").read_text(encoding="utf-8"))
        item = calibration["families"]["izhikevich"]
        feature_scale = item["feature_scale"]
        self._a = NengoNeuronFamilyAugmentor(
            family="izhikevich", limits=limits, dt=dt, beta=beta, mode="pes",
            learning_rate=learning_rate / feature_scale,          # <-- the one-line correction
            feature_tau=calibration["feature_tau_s"], feature_scale=feature_scale,
            native_current_scale=item.get("native_current_scale", 1.0), seed=seed)
```

10.2 Metrics used in Sec. VI-D

Effort (Table II): $\sqrt{\text{mean}_{k,i}(\delta_{k,i}/\ell_i)^2}$ of the *realized* residual (after the torque box), arithmetic mean over missions. **Rail fraction**: share of samples, $t \geq 2\,\text{s}$, all three axes, with $|\delta_{k,i}| \geq 0.98\,\beta\ell_i$ (corrected Izhikevich: 0–1.7%). **Activity occupancy**: share of unit-samples with $a_j > 10^{-6}$, mission mean per family and platform (LIF, ALIF, LIFRate: 3.2–4.4%; CMG, Izhikevich: $\geq 99.9\%$).

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 628–629 normalized RMS used for every effort value

```
    def normalized_rms(values: np.ndarray) -> float:
        return float(np.sqrt(np.mean((values / limits[None, :]) ** 2)))
```

11 Protocol and metrics (Sec. VI-A, VI-E)

11.1 Error metric and reductions

$\bar{\theta}_q = \frac{1}{N}\sum_k \theta_{q,k}$ over all $N = 90{,}000$ control steps (45 s at $h = 5 \times 10^{-4}\,\text{s}$), computed from the *true* attitude even when the controller sees noisy feedback. Reduction $= 100(\bar{\theta}_{q,\text{PD}} - \bar{\theta}_{q,\text{CMG}})/\bar{\theta}_{q,\text{PD}}$. The five-mission summary is the reduction of geometric means, $100\,(1 - \text{gm}(\bar{\theta}_{\text{CMG}})/\,\text{gm}(\bar{\theta}_{\text{PD}}))$, computed from unrounded values.

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 440–441 per-step true and fed-back error

```
            qerr_true = quat_angle(qe_true)
            qerr_feedback = quat_angle(qe_feedback)
```

experiments/final_robustness_gating_2026-09-20/run_final_robustness.py : 725–731 summary metrics

```
        "metrics": {
            "qerr_true_mean_rad": float(np.mean(qerr_true)),
            "qerr_true_max_rad": float(np.max(qerr_true)),
            "qerr_true_rmse_rad": float(np.sqrt(np.mean(qerr_true**2))),
            "qerr_feedback_mean_rad": float(np.mean(qerr_feedback)),
            "qerr_feedback_max_rad": float(np.max(qerr_feedback)),
            "qerr_feedback_rmse_rad": float(np.sqrt(np.mean(qerr_feedback**2))),
```

11.2 Measurement noise

Independent zero-mean Gaussian Euler angles form a small quaternion left-multiplied into q; Gaussian samples are added to each rate component (moderate 0.010 rad, 0.025 rad/s, 5 seeds; realistic 0.020 rad, 0.050 rad/s, 10 seeds). Both arms of a pair use the same seed (PCG64), hence the same noise stream (listing in Sec. 3, lines 328–334).

11.3 Repeated disturbances

$\boldsymbol{\tau}_{\mathrm{d}} = d_b \varrho(t)[\sin(1.31t + .2),\ .8\sin(1.73t + 1.1),\ .6\sin(2.17t + 2)]^{\mathsf{T}}$ in body axes, applied as the world-frame torque $R\boldsymbol{\tau}_{\mathrm{d}}$. ϱ is zero for a 5-s warm-up, then alternates 5-s on/off with raised-cosine edges $\frac{1}{2}(1 - \cos(\pi\xi/0.25))$.

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` : 110–129 envelope $\varrho(t)$

```
110 def disturbance_gain(t: float, args: argparse.Namespace) -> float:
111     """Return the deterministic continuous or intermittent disturbance gain."""
112     if args.disturbance_schedule == "continuous":
113         return 1.0
114     if t < args.disturbance_warmup:
115         return 0.0
116     phase = (t - args.disturbance_warmup) % (
117         args.disturbance_on + args.disturbance_off
118     )
119     if phase >= args.disturbance_on:
120         return 0.0
121     ramp = args.disturbance_ramp
122     if ramp <= 0.0:
123         return 1.0
124     if phase < ramp:
125         return float(0.5 * (1.0 - np.cos(np.pi * phase / ramp)))
126     if phase > args.disturbance_on - ramp:
127         x = (args.disturbance_on - phase) / ramp
128         return float(0.5 * (1.0 - np.cos(np.pi * x)))
129     return 1.0
```

`experiments/final_robustness_gating_2026-09-20/run_final_robustness.py` : 424–438 body-frame multi-sine, applied through `xfrc_applied`

```
424             disturbance_scale = disturbance_gain(t, args)
425             disturbance_body = np.zeros(3)
426             if args.disturbance_torque > 0.0 and disturbance_scale > 0.0:
427                 disturbance_body = disturbance_scale * args.disturbance_torque * np.array(
428                     [
429                         np.sin(1.31 * t + 0.2),
430                         0.8 * np.sin(1.73 * t + 1.1),
431                         0.6 * np.sin(2.17 * t + 2.0),
432                     ]
433                 )
434 
435             data.ctrl[:] = 0.0
436             data.xfrc_applied[:] = 0.0
437             data.ctrl[act_ids] = actuator_command
438             data.xfrc_applied[body_id, 3:6] = rotation @ disturbance_body
```

12 Changes relative to the September 12 companion

1. Equation numbering follows the current manuscript (30 equations, Propositions 1–2). The local-stability theorem and its $\beta_{\max}$ certificate are no longer in the paper and are removed here.

2. Added: the desired-rate feedforward P-(15) with the derivation of $\boldsymbol{\omega}_d$ and the transport bound (Sec. 3.5); the $\rho = 3$ baseline and its backstepping identity; the saturation-aware gate P-(28) with

its code and implementation tolerance; the equal-rate PES normalization for neuron families; the effort, rail and occupancy definitions.

3. Code references move from the archived `run_mujoco_nef.py` to the runner that produced the current numbers, `run_final_robustness.py`; the archived runner is cited only for the fixed-PD and $\rho = 1$ rows.

4. The occupancy table of the old Sec. 5.4 (archived campaign, $\leq 0.1\%$) is replaced by the corrected-protocol values.

5. The stiffness ratio is given for all three baselines, not only the fixed PD.

A Source files and hashes

Hashes of the files quoted in this document, compared with the `source_sha256` and `model_sha256` fields of the manifests of the published primary runs (X2 Hard45 and CF2 $\beta = .2$ Hard45, gated). "Not recorded" means the primary runs do not import that file; it is still quoted at the hash shown.

File	SHA-256	Provenance check
`src/gra_control/math3d.py`	`31620612f184a22f15971b1a7d15548f f2997b09a093254ab71ddb289e502c5e`	matches X2/CF2 run manifest
`src/gra_control/gate.py`	`ea6d7853fff724d2a6c6fd15afb9a18c f7f9cdae19ce6f04bcf4a5e2300a5e53`	matches X2/CF2 run manifest
`src/gra_control/platforms/drone.py`	`50af6d1f6de51788f727074c57b90ff8 5a3dc2a8688b91cdc49ee50623a123d9`	matches X2/CF2 run manifest
`experiments/nengo_nef_replication_ 2026-08-29/nengo_cmg.py`	`fbfd069d78d4040e738a8df702acb35a 08b5141e34a7cdb70c9d4e954d782562`	matches X2/CF2 run manifest
`experiments/nengo_nef_replication_ 2026-08-29/run_mujoco_nef_ baselines.py`	`3c12f663e4034fd94308f5d6293afaed a4ee2ad94cd590b57574b3b168b278e6`	matches X2/CF2 run manifest
`experiments/nengo_nef_replication_ 2026-08-29/run_mujoco_nef.py`	`10e50c274010aa9769dd73787a0ce5df e6e9294bc7bc91f70b36dc2012bc73d4`	not recorded by the primary runs
`experiments/nengo_nef_replication_ 2026-08-29/nengo_neuron_family.py`	`882ea82ee3101a514695ec8cb420945a 3d6424409271043323bd569d55d7d5e6`	not recorded by the primary runs
`experiments/nengo_nef_replication_ 2026-08-29/verify_archived_ equivalence.py`	`69af1748a3f4be2930d110634a9821af 48aa66ddd94d3c687899618f4bc05cc9`	not recorded by the primary runs
`experiments/final_robustness_ gating_2026-09-20/run_final_ robustness.py`	`e4f076565dc658017b78e40dbc5e384d 052f5d54be0914fbd33ffcdb6538a292`	matches X2 run manifest
`experiments/final_robustness_ gating_2026-09-20/run_a2_ robustness.py`	`cca415bbc9255ef10b15b80d0f1415b4 3333321bd9616c8c57c0e6d176c3b66c`	matches CF2 run manifest
`experiments/final_robustness_ gating_2026-09-20/run_controllers_ gated.py`	`48d934309a3f99b8861a72adb97cc78f 1e8236f93bc0b44b4c4e33f0d7ad133a`	not recorded by the primary runs
`experiments/final_robustness_ gating_2026-09-20/run_izh_ corrected.py`	`a80db1ef88feed8ecbce93c11dd65db6 be8eb267959d4f7a52f5a13ad25677e1`	not recorded by the primary runs
`experiments/final_robustness_ gating_2026-09-20/izh_adapter_ corrected.py`	`b7f6fbf16bee6c0c2ccdbaf19901ad6d 0c4a94fde940244beadc0f6e6a45409c`	not recorded by the primary runs

File	SHA-256	Provenance check
experiments/final_robustness_gating_2026-09-20/build_final_csv.py	cb5435fdd1cfe70e3705c351d6edda53b4b276ed0faa2840ac47642b453579d5	not recorded by the primary runs
experiments/final_robustness_gating_2026-09-20/make_ladder_figure.py	4cdad2bd5c6c2b9cb63933ddaddcdf20d40ad5f4d272a973994b92deee71fb5c	not recorded by the primary runs
experiments/neuron_basis_decisive_2026-09-24/run_neuron_basis.py	4861efc6c976b618d8e0470986e951fce4544520ee2b816ddebfed9d762cee52	not recorded by the primary runs
experiments/neuron_basis_decisive_2026-09-24/family_basis_adapter.py	20331a21fb0d37d45748445b39c8e60488713055c8735581d3abadd115444516	not recorded by the primary runs
experiments/theory_certificate_2026-09-12/compute_local_certificate.py	b7eb811f74600c6deeddcd7b5a8a634d02d889d48b1d6035436b6dfdd3793e60	not recorded by the primary runs
reproduce/verify_acc2027_claims.py	52e73314db52fe8c8b696ba664301e131a2378e429095bb376d24b9935a82df4	not recorded by the primary runs
reproduce/check_rate_frame.py	6c175f3e5a06c5b75dfd19b19706cd1f7b9142c6b37d32aeb3aeb01c695f7796	not recorded by the primary runs
third_party/mujoco_menagerie/skydio_x2/scene.xml	d932ffbe9f0d1fa4bcfe5746d4cfc0c5652960f00223fdaf2ffd6aff6b217c97	matches X2 run manifest
models/cf2_rotor_design/scene.xml	410e0975d066abb1ad726415250c7894 8c0f25b2f99f3b73746c161603dfdaf6	matches CF2 run manifest

B Recomputed paper numbers

Output of `reproduce/verify_acc2027_claims.py`: each number the verifier reads from the manuscript, recomputed from the raw run outputs listed in `REPRODUCE_ACC2027.md`.

#	Paper location	Claim	Paper value	Recomputed	Status
1	VI-B primary	x2 gated geo reduction	37.3	37.3	OK
2	VI-B primary	cf2-rotor gated geo reduction	22.4	22.4	OK
3	VI-B primary	x2 geo error PD+ff -> CMG (1e-3 rad)	4.54->2.85	4.54->2.85	OK
4	VI-B primary	cf2-rotor geo error PD+ff -> CMG (1e-3 rad)	3.77->2.92	3.77->2.92	OK
5	VI-B primary	CF2 ungated beta=.05 geo	6.5	6.5	OK
6	VI-B primary	X2 ungated geo	35.6	35.6	OK
7	VI-B primary	Full3/Hard45 reductions X2	12.4/13.8	12.4/13.8	OK
8	VI-B primary	Full3/Hard45 reductions CF2	5.4/7.3	5.4/7.3	OK
9	Table II rho=1	x2 geo reduction on PD+ff rho=1	57.8	57.8	OK
10	Table II rho=1	cf2-rotor geo reduction on PD+ff rho=1	57.6	57.6	OK
11	Table II fixed PD	x2 geo reduction on fixed PD	44.5	44.5	OK
12	Table II fixed PD	cf2-rotor geo reduction on fixed PD	44.2	44.2	OK
13	VI-B primary	fixed-PD range, all ten cells	17.3-60.6	17.3-60.6	OK
14	VI-B portability	lqr	31.1/22.4	31.1/22.4	OK
15	VI-B portability	pid	36.7/22.4	36.7/22.4	OK
16	VI-B portability	smc	43.8/26.6	43.8/26.6	OK
17	VI-B portability	mpc	33.4/22.7	33.4/22.7	OK
18	VI-B portability	nmpc	31.9/21.7	31.9/21.7	OK
19	VI-B portability	backstepping	37.3/22.4	37.3/22.4	OK
20	Table II Izhikevich	x2 Delta row (5 missions, geo)	-48.0,5.8,4.5,-1.1,-76.3,-18.9	-48.0,5.8,4.5,-1.1,-76.3,-18.9	OK
21	Table II Izhikevich	cf2-rotor Delta row (5 missions, geo)	0.7,1.3,0.8,-0.5,-33.3,-5.4	0.7,1.3,0.8,-0.5,-33.3,-5.4	OK
22	Abstract / VI-D	Izhikevich geo increase X2/CF2	18.9/5.4	18.9/5.4	OK
23	VI-D	Izhikevich geo error ratio X2/CF2	1.19/1.05	1.19/1.05	OK
24	VI-D	Izhikevich Full3/Hard45 X2	5.8/4.5	5.8/4.5	OK
25	Table II Izhikevich	x2 effort row	0.053,0.051,0.053,0.106,0.052,0.063	0.053,0.051,0.053,0.106,0.052,0.063	OK

#	Paper location	Claim	Paper value	Recomputed	Status
26	Table II Izhikevich	cf2-rotor effort row	0.012,0.021,0.021,0.047, 0.012,0.023	0.012,0.021,0.021,0.047, 0.012,0.023	OK
27	Table II Izhikevich	x2 CMG effort row	0.003,0.023,0.023,0.109, 0.001,0.032	0.003,0.023,0.023,0.109, 0.001,0.032	OK
28	Table II Izhikevich	cf2-rotor CMG effort row	0.005,0.010,0.012,0.049, <0.001,0.015	0.005,0.010,0.012,0.049, <0.001,0.015	OK
29	VI-D	Izhikevich rail fraction range (%)	0-1.7	0-1.7	OK
30	Table II Izhikevich	x2 Delta row, fixed PD	39.6,37.5,32.2,6.6,46.6, 33.8	39.6,37.5,32.2,6.6,46.6, 33.8	OK
31	Table II Izhikevich	cf2-rotor Delta row, fixed PD	33.7,38.9,32.2,diverges, 41.5,–	33.7,38.9,32.2,diverges, 41.5,–	OK
32	VI-D alternatives	paired-current	11.9/4.8	11.9/4.8	OK
33	VI-D alternatives	constant-one	10.3/4.3	10.3/4.3	OK
34	VI-D alternatives	LIFRate threshold-aware, held out	15.8/11.4	15.8/11.4	OK
35	VI-D	LIF/ALIF/LIFRate active unit-samples, mission mean, rho=3 (%)	3.2-4.4	3.2-4.4	OK
36	VI-E noise	seeds moderate/realistic	5/10	5/10	OK
37	VI-E noise	realistic Full3/Hard45 X2	11.3/10.8	11.3/10.8	OK
38	VI-E noise	realistic Full3/Hard45 CF2	3.9/4.0	3.9/4.0	OK
39	VI-E noise	Roll/Yaw360 change range	-5.3..2.2	-5.3..2.2	OK
40	VI-E noise	realistic geo X2/CF2	2.3/1.0	2.3/1.0	OK
41	VI-E noise	CF2 beta=.5 Hardstep diverges	15/15	15/15	OK
42	VI-E noise	rotor-bound share (%)	61-79	61-79	OK
43	VI-E disturbance	x2 burst reduction 1x	14.4	14.4	OK
44	VI-E disturbance	cf2-rotor burst reduction 1x	5.9	5.9	OK
45	VI-E disturbance	x2 burst reduction 0.5x/2x	24.0/13.3	24.0/13.3	OK
46	VI-E disturbance	cf2-rotor burst reduction 0.5x/2x	6.5/4.7	6.5/4.7	OK
47	VI-E disturbance	x2 burst error (deg)	1.61->1.38	1.61->1.38	OK
48	VI-E disturbance	cf2-rotor burst error (deg)	2.27->2.14	2.27->2.14	OK
49	VI-E disturbance	x2 quiet reduction 1x	14.7	14.7	OK
50	VI-E disturbance	cf2-rotor quiet reduction 1x	19.8	19.8	OK
51	VI-E disturbance	X2 quiet 0.5x (deg)	0.021->0.036	0.021->0.036	OK
52	VI-E disturbance	CF2 burst 1x at beta=.5 / beta=.05	13.7/1.5	13.7/1.5	OK
53	VI-F Hardstep	X2 ungated / gated	-17.0/-2.0	-17.0/-2.0	OK
54	VI-F Hardstep	CF2 beta=.5 gated Hardstep / geo	-1.2/36.5	-1.2/36.5	OK
55	VI-F Hardstep	CF2 beta=.5 ungated divergence (deg, rotor %)	136/92	136/92	OK
56	VI-F Hardstep	gate blocked share X2/CF2 (%)	20.5/15.5	20.5/15.5	OK
57	VI-A	Nengo vs NumPy worst relative error over ten cases	2.013e-06 (10)	2.013e-06 (10)	OK
58	VI-C ablation	PES corner (rad/s)	9.60	9.60	OK
59	Fig. 4	legend degrees PD/CMG per level	16.85/7.91,4.07/2.19, 0.60/0.52	16.85/7.91,4.07/2.19, 0.60/0.52	OK

References

[1] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," *Matrix*, vol. 58, no. 15–16, pp. 1–35, 2006.

[2] H. Schaub and J. L. Junkins, *Analytical Mechanics of Space Systems*, 4th ed. AIAA, 2018.

[3] S. P. Bhat and D. S. Bernstein, "A topological obstruction to continuous global stabilization of rotational motion and the unwinding phenomenon," *Systems & Control Letters*, vol. 39, no. 1, pp. 63–70, 2000.

[4] N. A. Chaturvedi, A. K. Sanyal, and N. H. McClamroch, "Rigid-body attitude control," *IEEE Control Systems Magazine*, vol. 31, no. 3, pp. 30–51, 2011.

[5] C. G. Mayhew, R. G. Sanfelice, and A. R. Teel, "Quaternion-based hybrid control for robust global attitude tracking," *IEEE Transactions on Automatic Control*, vol. 56, no. 11, pp. 2555–2566, 2011.

[6] I. Rubio Scola, O. A. García Alcántara, S. Sandoval, E. S. Espinoza Quesada, H. Haimovich, and L. R. Garcia Carrillo, "Globally asymptotically stable trajectory tracking of underactuated UAVs using geometric algebra," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 62, pp. 1491–1508, 2026.

[7] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[8] Google DeepMind, "MuJoCo Menagerie," https://github.com/google-deepmind/mujoco_menagerie, model repository; accessed Sep. 5, 2026.

[9] L. R. García Carrillo, A. E. Dzul López, R. Lozano, and C. Pégard, *Quad Rotorcraft Control: Vision-Based Hovering and Navigation*, ser. Advances in Industrial Control. London: Springer, 2013.

[10] J. C. Monteiro, F. Lizarralde, and L. Hsu, "Optimal control allocation of quadrotor UAVs subject to actuator constraints," in *2016 American Control Conference (ACC)*, 2016, pp. 500–505.

[11] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, 1952.

[12] M. Hochbruck and A. Ostermann, "Exponential integrators," *Acta Numerica*, vol. 19, pp. 209–286, 2010.

[13] D. MacNeil and C. Eliasmith, "Fine-tuning and the stability of recurrent neural networks," *PLoS ONE*, vol. 6, no. 9, p. e22885, 2011.

[14] A. R. Voelker, "A solution to the dynamics of the prescribed error sensitivity learning rule," Centre for Theoretical Neuroscience, University of Waterloo, Tech. Rep. CTN-TR-20151001-01, Oct. 2015. [Online]. Available: https://www.compneuro.uwaterloo.ca/files/publications/voelker.2015.pdf

[15] P. A. Ioannou and P. V. Kokotović, "Instability analysis and improvement of robustness of adaptive control," *Automatica*, vol. 20, no. 5, pp. 583–594, 1984.

[16] P. A. Ioannou and J. Sun, *Robust Adaptive Control*. Upper Saddle River, NJ: Prentice Hall, 1996.

[17] T. Bekolay, J. Bergstra, E. Hunsberger, T. DeWolf, T. C. Stewart, D. Rasmussen, X. Choo, A. R. Voelker, and C. Eliasmith, "Nengo: A Python tool for building large-scale functional brain models," *Frontiers in Neuroinformatics*, vol. 7, p. 48, 2014.

[18] H. H. Bauschke and P. L. Combettes, *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, 2nd ed. Cham: Springer, 2017.

[19] S. Sandoval and P. L. De Leon, "The instantaneous spectrum: A general framework for time-frequency analysis," *IEEE Transactions on Signal Processing*, vol. 66, no. 21, pp. 5679–5693, 2018.

[20] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Transactions on Neural Networks*, vol. 14, no. 6, pp. 1569–1572, 2003.